

SISTEMA À PROVA DE FALHAS

failproof

Um padrão próprio de busca de erros e correções para sessões Claude Code — o modelo aprende com cada falha, é impedido de declarar sucesso sem prova, e se recupera sozinho quando roda sem supervisão.

Autor: Jessé Freitas — OmniForge

Projeto: OmniRift · `tools/failproof/`

Versão do guia: 1.0 · 05 de julho de 2026

Stack: Python 3.10+ (stdlib only) · SQLite/FTS5 · hooks Claude Code

Status: 67 testes verdes · PR #92

100% standalone — OmniMemory opcional

Sumário

01 O que é e por que existe

O problema das falhas silenciosas e do "achei que tinha corrigido"

02 Como funciona — visão geral

As três camadas e o princípio de falha isolada

03 Camada 1 — Failbase (o cérebro)

Onde os erros e correções ficam guardados

04 Camada 2 — Hooks (a disciplina)

Os quatro pontos de interceptação automática

05 Camada 3 — Watchdog (a rede de segurança)

Auto-recuperação de sessões autônomas

06 Instalação

Um comando, portátil, sem quebrar nada

07 Uso no dia a dia

CLI, captura de CI e o ciclo de aprendizado

08 Plugins opcionais

Notificações e sincronização com o cluster

09 Manutenção e desinstalação

Como inspecionar, atualizar e remover

01 O que é e por que existe

O **failproof** transforma qualquer sessão do Claude Code (especialmente o Opus) num agente que segue um padrão sistemático diante de erros: consulta o que já sabe antes de debugar, registra toda falha resolvida, é impedido de mentir sobre "estar corrigido", e — quando roda sozinho — se recupera de travamentos sem precisar de você.

Ele ataca três modos de falha que se repetem em agentes de código:

Modo de falha	O que o failproof faz
Re-descobrir o mesmo erro — o agente gasta tempo diagnosticando algo que já resolveu antes.	Toda falha resolvida vira registro. Da próxima vez que o mesmo erro aparecer, o fix conhecido é entregue na hora, sem re-diagnóstico.
Falso "corrigido" — o agente declara sucesso sem ter rodado nada que prove.	Um portão intercepta a conclusão: se há afirmação de sucesso mas nenhuma execução verde após a última edição, ele exige a validação real.
Travar sem supervisão — uma sessão autônoma entra em loop e ninguém percebe.	Um vigia externo detecta a estagnação e escala a recuperação em três níveis, terminando em notificação a você.

Princípio de portabilidade

O núcleo é **100% standalone**: Python da biblioteca padrão, um arquivo SQLite local, zero dependência de rede ou de infraestrutura. Funciona idêntico numa máquina de cliente que não tem nada do ecossistema OmniForge. As integrações (cluster, notificações) são **plugins detectados em tempo de execução** — se o ambiente tiver, ligam sozinhos; se não, somem sem quebrar nada.

02 Como funciona — visão geral

São três camadas que se comunicam apenas através da base de dados, e cada uma **falha de forma isolada** — a propriedade que dá nome ao sistema. Um hook que dê erro nunca derruba a sessão; uma sincronização que atrase não bloqueia o trabalho; o vigia roda fora da sessão que ele observa.

Camada 1 · Failbase **O CÉREBRO**

Um banco SQLite local (`~/.claude/failbase/failbase.db`) onde cada erro vira um registro estruturado: sintoma, causa, correção validada e um contador de reincidência. Uma assinatura normalizada permite reconhecer "esse erro eu já vi" em milissegundos.

Camada 2 · Hooks **A DISCIPLINA**

Quatro pontos onde o Claude Code chama um script do failproof automaticamente: depois de rodar comandos, quando você corrige o agente, quando ele tenta concluir, e no início de cada sessão. Todos escrevem e leem da Failbase, e todos são à prova de erro (falham-aberto).

Camada 3 · Watchdog **A REDE DE SEGURANÇA**

Um processo externo, disparado a cada 5 minutos, que vigia as sessões autônomas registradas. Detecta estagnação ou loop e escala a recuperação em três níveis progressivos, alimentando a Failbase com um postmortem no fim.

O ciclo de aprendizado

- **Erro acontece** — um comando falha, você corrige o agente, ou o CI fica vermelho.
- **Captura automática** — o hook correspondente grava o par (sintoma → correção) na Failbase, marcando como validado quando há prova de que funcionou.
- **Reuso instantâneo** — na próxima vez que o mesmo erro (mesma assinatura) surgir, o fix conhecido é injetado direto no resultado, sem o agente gastar tokens procurando.
- **Prevenção proativa** — ao abrir uma nova sessão no projeto, os erros mais recorrentes são injetados no contexto: "não repita estes".

03 Camada 1 — Failbase

É a única fonte de verdade compartilhada por todas as camadas. Cada registro guarda o essencial para reconhecer e resolver uma falha:

Campo	Papel
<code>signature</code>	Hash normalizado do erro (comando + texto, sem caminhos, números ou hashes voláteis). É o que permite casar "o mesmo erro" mesmo com detalhes diferentes.
<code>symptom</code> / <code>root_cause</code> / <code>fix</code>	O erro cru, o diagnóstico e a correção que funcionou.
<code>fix_validated</code>	Marca se a correção foi confirmada por uma execução verde real.
<code>source</code>	De onde veio: <code>session</code> , <code>human-feedback</code> , <code>ci</code> ou <code>watchdog</code> .
<code>hits</code>	Reincidência — um erro que volta sobe no ranking e ganha prioridade.

Sem o cluster, a busca usa o **FTS5** nativo do SQLite (encontra "erro parecido com este"). Com o plugin de sincronização, ganha busca semântica por embedding no pgvector. A CLI dá acesso direto:

```
# registrar um erro e sua correção
python3 ~/.claude/failbase/failbase.py add \
  --symptom "psycopg2 connection refused host pg" \
  --fix "usar service name core-net na rede overlay" \
  --source human-feedback --project omnirift --validated

# procurar por texto
python3 ~/.claude/failbase/failbase.py search "connection refused"

# ver o estado da base
python3 ~/.claude/failbase/failbase.py stats
```

04 Camada 2 — Hooks

Quatro scripts que o Claude Code invoca automaticamente. Todos têm um envelope `try/except` que garante saída limpa mesmo com entrada corrompida — **um hook nunca trava a sessão** (invariante testado contra entradas adversariais).

PostToolUse — captura de par falha→fix

Depois de cada comando no shell, mantém um rastro da sessão. Quando um comando falha e outro parecido passa em seguida, grava o par automaticamente como correção validada. E quando um comando falha com uma assinatura já conhecida, injeta o fix no resultado:

```
💡 Failbase: erro conhecido (visto 4x). Fix que funcionou antes:  
docker network create --driver overlay core-net
```

UserPromptSubmit — detector de correção humana

Quando você corrige o agente ("está errado", "não é assim", "regressão", "de novo isso"), o hook instrui o modelo a registrar na Failbase o que ele fez de errado e o entendimento correto — o gatilho é determinístico, mas o conteúdo do registro é do modelo, porque só ele sabe a causa raiz.

Stop — portão de evidência

Quando o agente tenta concluir, varre o final da conversa: se há afirmação de sucesso ("corrigido", "funcionando", "pronto") mas **nenhuma execução verde após a última edição de arquivo**, bloqueia com um pedido de validação real.

Anti-loop embutido

O portão bloqueia **no máximo uma vez por turno**. Se o modelo insistir, a segunda passagem libera (mas marca o placar). Isso impossibilita, por construção, um loop infinito de bloqueio — a segurança do próprio mecanismo de segurança.

SessionStart — erros conhecidos do projeto

No início de cada sessão, injeta os 10 erros mais recorrentes do projeto atual (ranqueados por reincidência × recência), num bloco compacto de ~500 tokens: "erros já conhecidos neste projeto — não repita".

05 Camada 3 — Watchdog

Para sessões que rodam sem você olhando (agendadas, em fila, autônomas). Elas se registram criando um arquivo em `~/.claude/failbase/watch/<id>.json`:

```
{
  "session_id": "s1",
  "transcript_path": "/caminho/transcript.jsonl",
  "pid": 12345,
  "relaunch_cmd": "claude -p \"$(cat {postmortem})\" ..."
}
```

A cada 5 minutos (via systemd timer ou cron), o vigia checa cada sessão. Se o transcript não muda há mais de 20 minutos, ou se o mesmo erro se repete 3 vezes seguidas, escala:

Strike	Ação
1	Injeta na sessão o erro detectado + os fixes conhecidos da Failbase e pede uma estratégia diferente (ou reinicia com o prompt aumentado).
2	Encerra a sessão, gera um postmortem automático (últimas ações, erro repetido, o que já foi tentado) e reinicia uma sessão nova com esse postmortem no contexto.
3	Para de vez, grava o postmortem na Failbase e notifica você (ntfy/Telegram, ou um arquivo de alerta se não houver canal configurado).

Recuperação com teto

A escala é autônoma mas limitada — nunca queima recursos indefinidamente. Todo postmortem alimenta a base, então a próxima sessão que pegar uma tarefa parecida já nasce sabendo o que não funciona.

06 Instalação

Um comando, a partir do diretório do projeto:

```
cd tools/failproof
./install.sh
```

O instalador é **portátil e não-destrutivo**. Ele:

- Cria a base e os diretórios em `~/.claude/failbase/` ;
- Copia os 4 hooks para `~/.claude/hooks/` com o prefixo `failproof_` ;
- Registra os hooks no `~/.claude/settings.json` por **merge idempotente** — preserva suas configurações e outros hooks existentes, e rodar duas vezes não duplica nada;
- Ativa o watchdog via systemd user timer; se o systemd não estiver disponível, imprime a linha de cron para você adicionar;
- Detecta os plugins viáveis pelo ambiente e liga só os que existem.

Requisito único

Python 3.10 ou superior. Nada mais — nem `pip install` , nem serviço externo. Os hooks entram em vigor na próxima sessão do Claude Code.

Para confirmar que ficou tudo no lugar:

```
systemctl --user is-active failproof-watchdog.timer    # -> active
python3 ~/.claude/failbase/failbase.py stats          # -> {"total": 0, ...}
```


07 Uso no dia a dia

Depois de instalado, o failproof trabalha sozinho — a captura e o reuso acontecem em segundo plano. O que você pode fazer manualmente:

Consultar e alimentar a base

Use a CLI (seção 3) para procurar um erro, registrar uma correção que descobriu fora do fluxo automático, ou ver o placar com `stats`.

Integrar ao CI

Dois subcomandos, chamados em pontos distintos do pipeline. O `red` registra o job que falhou; o `green` fecha o par quando o mesmo job (mesmo `--branch`) volta a passar, gravando o diff da correção:

```
# quando o job falhou
python3 ~/.claude/failbase/failbase_ci.py red \
  --job pytest --branch "$BRANCH" --log out.log --project omnirift

# quando o mesmo job passou depois
python3 ~/.claude/failbase/failbase_ci.py green \
  --job pytest --branch "$BRANCH" --diff fix.diff --project omnirift
```

Um `green` sem um `red` pendente para aquele par é simplesmente ignorado. Funciona standalone, sem nenhum acesso ao cluster.

Ver o que o agente já sabe

Toda vez que você abre uma sessão nova num projeto que já acumulou erros, o bloco de "erros conhecidos" aparece no contexto inicial. É o sinal de que o sistema está aprendendo.

08 Plugins opcionais

Ligam sozinhos quando a variável de ambiente correspondente existe. Ausência nunca quebra a base — apenas desliga o recurso.

Plugin	Ativa com	Comportamento
notify	<code>FAILPROOF_NTFY_URL</code> ou <code>FAILPROOF_TELEGRAM_TOKEN</code> + <code>FAILPROOF_TELEGRAM_CHAT</code>	Envia os alertas de strike 3 do watchdog. Sem nenhum canal, grava em <code>~/.claude/failbase/alerts/</code> .
sync-omnimemory	<code>FAILPROOF_SYNC_CMD</code>	Espelha a Failbase para o pgvector do cluster (busca semântica cross-máquina). O comando recebe os registros novos por stdin e, se sair com sucesso, eles são marcados como sincronizados.

```
# exemplo: notificação por Telegram
export FAILPROOF_TELEGRAM_TOKEN="123:ABC"
export FAILPROOF_TELEGRAM_CHAT="-9876543210"
```

Segurança

O comando de sincronização vem **exclusivamente** da variável de ambiente — nunca de dados da Failbase — e os registros trafegam só por stdin. Nenhum token ou URL é impresso em log.

09 Manutenção e desinstalação

Inspecionar

```
python3 ~/.claude/failbase/failbase.py stats          # placar
python3 ~/.claude/failbase/failbase.py export         # dump JSONL de tudo
systemctl --user list-timers failproof-watchdog.timer # próximo tick do vigia
ls ~/.claude/failbase/postmortems/                   # relatórios do watchdog
```

Atualizar

Rodar `./install.sh` de novo é seguro — o merge é idempotente e os arquivos são sobrescritos pelas versões novas. O banco de dados é preservado.

Desinstalar

```
cd tools/failproof
./uninstall.sh
```

Remove os hooks, as entradas no `settings.json` e o timer — mas **preserva o banco** (`~/.claude/failbase/failbase.db`), para que o conhecimento acumulado sobreviva a uma reinstalação.

Referências no repositório

Especificação de projeto: `docs/superpowers/specs/2026-07-05-failproof-design.md`
Plano de implementação (12 tarefas TDD): `docs/superpowers/plans/2026-07-05-failproof.md`
Código e testes: `tools/failproof/` · Pull Request: **#92** em `jesse_freitas/maestri_linux`